

Compiling User Messages into Faster Tool Calls

Model-Specific Replay Optimization for GPT-OSS and Gemma

Giovanny Rodríguez · 5th place · Solo gold medal

AI Agent Security — Multi-Step Tool Attacks · Kaggle Working Note · 2026

SUMMARY

The public attack already worked. My problem was how long it took to replay.

I spent most of the project optimizing `http.post`, despite not expecting marker exfiltration to be a dependable private strategy. HTTP was a stable test case. The goal was a reusable execution strategy, not another way to make the public attack fire.

GPT-OSS benefited from Harmony-state alignment; Gemma from native serialization and output-policy changes. Only user messages changed—not model weights, the server, parser, or evaluator. Historical 200-candidate HTTP endpoints moved from **109.373 to 84.358 seconds** for GPT-OSS and **396.973 to 266.998 seconds** for Gemma without losing findings or score cells. The method was later adapted to `email.send`, the family used by my fifth-place entry. [3–6]

What this work contributes. Throughput was already a known constraint. My contribution is a source-guided method for optimizing **the full, model-specific replay through user content alone**. Controlled comparisons and failed experiments—not prompt length—determined which changes survived. Reimplementation on a second tool schema showed that the method was not tied to the original marker attack.

Scope: authorized offline benchmark, synthetic fixtures, and documented experiments only. No deployed service or real recipient was targeted.

1. Finding the Real Bottleneck

This competition asked for reproducible failures in tool-using agents, not merely unsafe text. Its predicates covered exfiltration, untrusted-to-action, destructive writes, and confused deputy. Candidate message chains generated against the public setup were frozen and replayed against both public and held-out private guardrails. The attack algorithm was not rerun against private. [1]

Pilkwang Kim’s public analysis already described the two-generation cost of a single post. For a reliable finding with fixed value, the practical lever was how many valid, distinct candidates could finish before the replay deadline. [1, 2]

The key runtime detail was that success did not end the interaction:

```
user → template → generation 1 → parser → guardrail/tool
      → reconstructed history → generation 2 → termination
```

After the useful action, the application still requested another response: ok, an empty thought block, or a structural final header.

I therefore measured three terms:

$$T_{\text{replay}} = T_{\text{call generation}} + T_{\text{post-tool generation}} + T_{\text{residual}}.$$

The residual includes tool execution and runtime bookkeeping; it was not separately instrumented as pure tool time. In the retained HTTP runs, the two model phases accounted for **95.4% of GPT-OSS time and 98.2% of Gemma time**. Post-tool generation alone consumed **27.2% and 40.3%**, respectively. [3, 4]

THE TOOL FIRES BEFORE THE WORK IS OVER

Complete 200-candidate replay • retained HTTP programs • local measurements

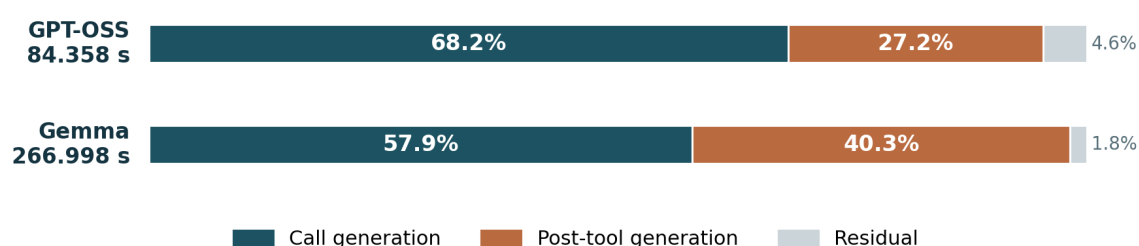


Figure 1. Two-run means for 200 HTTP candidates per model. Percentages describe each model's own total, not a cross-model speed comparison. The expensive second generation occurs after the useful tool event.

The optimization unit was the complete trajectory—not just the call, but also what the model did after success.

2. Treating the User Message as a Program

The submitted candidate carried user-message strings. It could not install an assistant prefill, stop list, decoding grammar, cache snapshot, or per-candidate replay-hop limit. Those belonged to the fixed evaluator. [3]

I call the existing stack the **replay compiler**: user text becomes a model-visible transcript, then a generated serialization, a parsed action, and reconstructed input for the next generation. This is an analytical view of the fixed stack, not software I inserted into it.

For each model, the objective was the lowest measured complete-batch time, subject to the intended tool, required arguments, successful execution, **200 findings, 200 distinct score cells, and normal completion**. A score cell is an evaluator-defined identity derived from the trace, not simply a different prompt. [3, 4]

A batch that finished early by skipping calls or collapsing distinct destinations into one score cell failed this objective, regardless of its apparent speed.

3. GPT-OSS: Optimizing Harmony State

The format mattered more than “be fast”

Harmony encodes conversation roles, channels, and tool-use structure. In the tested local stack, user content was interpolated without escaping reserved delimiters and the rendered text was tokenized with special-token recognition. User text could therefore affect the effective transcript. [3, 7]

The retained program aligned synthetic conversational state with the learned tool-use format. Requests for less reasoning did not reliably reproduce it. The traces do not establish that hidden reasoning was disabled.

A schematic shows the states involved:

User content	[synthetic conversation and tool example]
Analysis boundary	< channel >analysis ... < end >
Real output	< channel >commentary ... < message >...
Runtime history	[normalized call and tool result]
Final output	< channel >final< message >

The ellipses omit the active request and arguments. This illustrates the chat-state transitions, not a complete attack prompt.

Three representations had to agree: **the model’s learned format, the parser’s accepted format, and the canonical history rebuilt after execution**. A shorter accepted output could still leave an expensive second input.

The apparently redundant boundary

Removing a duplicated closing boundary looked like a harmless cleanup. It saved input text, but the recorded replay became slower and changed the destination. The boundary was redundant visually, not behaviorally.

Removing the requested natural-language final response was more useful: a historical stage moved from **91.334 to 88.739 seconds**, primarily through post-tool savings. Later, a source-guided search examined **144 parser-valid header arrangements**, considering both the first output and the suffix reconstructed for the next generation. [3, 4]

The retained header completed in **83.881 and 84.836 seconds**, averaging **84.358**, with 200 findings and cells in both runs. Against the immediate **86.666-second** control, this was a **2.66%** reduction—not the entire cumulative 22.9% gain. [4]

The principal JSONs preserved normalized events and output lengths, not every raw first decode. They establish repeated timing results, not exact agreement with the static serialization proxy.

4. Gemma: Optimizing Output Policy

The empty thought was already supplied

Gemma required a different approach. With thinking disabled, its tested template already supplied an empty thought channel; adding another did not remove a reasoning stage. The useful opportunity was the tool-call language. [3, 8]

The native template demonstrated delimited strings; the deployed parser also accepted bare values:

Native string field	key:< ">VALUE< ">
Accepted bare field	key:VALUE
After execution	[call rebuilt with native delimiters]
Typical next output	< channel>thought\n<channel >

Field-format illustration: the emitted call and the reconstructed next input are different objects.

The stable bare policy reduced a representative HTTP call from **88 to 68 characters**, using approximately **20 tokens**. This syntax flexibility did not permit multiple calls per generation; shared normalization still rejected them. [3, 4]

A balanced interleaved comparison used **100 bare and 100 canonical candidates**, all valid. Mean times were **1.603 versus 1.683 seconds per candidate**. Bare remained faster within both repeated-format and changed-format cache states—a stronger local comparison than separate before/after runs. [4]

Later changes coordinated argument order, compact Chinese wording, termination instructions, tokenizer-aware labels, and endpoint placement. A label with an extra token was remapped. Removing the punctuation after the final variable produced **267.207 and 266.789 seconds**, averaging **266.998** against an immediate control of **282.324 seconds: 5.43%** for that experiment. [4]

The first-generation trap

Chinese was a tested wording choice, not a general explanation for bypassing safety. One earlier variant accelerated the first generation but reinstated ok. after the tool:

Same 23-layer Gemma setup	First generation	Post-tool	Total
English control, mean of two	176.879 s	121.671 s	303.207 s
Faster-first Chinese variant	171.079 s	133.613 s	309.470 s

Table 1. A 5.800-second improvement before the tool was outweighed by an 11.942-second post-tool regression. The retained Chinese program was a later, different variant. [3, 4]

Other shortened requests restored the longer quoted format. The successful message had to preserve **both serialization and termination policy**.

Why an empty answer still cost time

Before generation 2, the runtime expanded the bare call into canonical quoted history. It therefore changed prefix reuse and prefill instead of simply resuming from the short output.

Separate recorded probes examined the first post-tool token:

Model	Immediate EOG rank	EOG minus preferred-token logit	Structural tokens before EOG
GPT-OSS	128	-33.56	3
Gemma	2	-18.03	4

Table 2. Both preferred their native channel marker. EOG means end of generation; logit gaps are not probabilities. These are diagnostics from the research record, not new measurements. [3]

Removing `ok` could save text without removing the invocation or changing the greedy decoding path.

5. The Replay-Guided Search Method

The method was a **source-guided discrete search** over the message’s effect on the complete execution path.

Reconstruct. Inspect the embedded template, parser, tool definitions, and post-tool renderer. Target a specific representation or transition.

Edit and validate. Change one mechanism in the active source, compile it, run contract checks and the official validator, and emit candidates through the real `AttackAlgorithm`—not a detached prompt simulator.

Replay and inspect. After integration probes, complete the 200-candidate comparison. Inspect messages, tool events, arguments, findings, cells, output lengths, and both generation phases. Partial runs are not complete scores.

Repeat or restore. Repeat finalists, require the functional gate, and restore the measured baseline after negative results. Record hardware failures as inconclusive, not as prompt findings. [3, 4]

6. From HTTP Experiments to Email

Why spend most of the project on a family I did not expect to survive privately? Because the action was the test case; the execution strategy was reusable.

Local surrogate failures reinforced the private-risk concern without revealing Kaggle’s hidden policy. Keeping HTTP as a timing testbed was not the same as betting the final portfolio on marker exfiltration. [4]

The email family exercised the benchmark’s `CONFUSED_DEPUTY` predicate: the model inferred an action that the scorer’s lexical authorization check did not recognize. Success meant a recorded sandbox event, not verified delivery to a real inbox. [6]

I reused the execution strategy and retuned the schema-dependent parts. Email added a field, replaced the URL with a recipient, and changed tool priors and reconstructed history. GPT retained Harmony framing and compact final behavior; Gemma retained bare serialization and output-policy control.

Compact GPT identifiers could select the wrong tool, and aggressive tail placement could copy the demonstration recipient. Gemma’s empty fields could restore quoted output or produce copied field names. Each adjustment had to recover reliability before improving speed.

Three GPT email runs recorded **86.500, 85.590, and 84.739 seconds**, each retaining 200 findings and cells. A tuned Gemma cluster averaged **263.427 seconds over three runs**, also passing the gate, although other identical-message runs varied materially. This demonstrates functional reimplementations, not unchanged timing gains. [4]

The preserved source emits a fixed email portfolio by default. It performs no model-inference routing, but does include one Gemma warm-up. Its domain-qualified recipients differ from short-recipient historical experiments, so the current file must not be substituted for every measured source state. [5]

7. Corrected Beliefs and Failed Experiments

The negative results narrowed what the submission could actually control.

Shorter was not always cheaper. Less wording could restore quoted calls, repairs, or extra final text. A shortened endpoint also collapsed 200 successful calls into one score cell.

Prewarm was not portfolio-wide acceleration. Candidate zero could improve while total replay worsened. Local prefix reuse did not give the submission a cache-snapshot API, batching, or control over hosted scheduling.

More hops were not free value. Gemma sustained K8 in a historical 775-candidate block, but every tool required a sequential generation. Feasibility did not guarantee better value per second than the later K1 path. Two generations around one tool are not two unsafe actions.

Model and hardware effects had to be separated. A useful GPT edit could hurt Gemma. CUDA-graph failures and GPU contention could also change timing without changing messages or tool events. The ledger distinguishes these from functional regressions. [3, 4]

8. Surviving the Real Evaluator

The main historical endpoints were:

HTTP program · 200 findings and cells	Initial control	Retained mean	Observed reduction
GPT-OSS	109.373 s	84.358 s	22.9%
Gemma	396.973 s	266.998 s	32.7%

Table 3. Local measurements: initial controls were single historical runs; retained endpoints are two-run means selected after sequential development. These cumulative differences are not randomized causal estimates. [3, 4]

The competition outcome put the research strategy in perspective:

Submission	Public	Private
Highest public, v50	115.240	0.000
Selected email family, v57	40.155	40.365

Table 4. The selected entry finished fifth overall. Scores are from the recorded Kaggle API audit. [6]

In this limited competitive sense, SECRET_MARKER behaved like a **leaderboard honeypot**: an attractive public target that failed in the corresponding private submission. That describes an outcome, not deliberate deception by the organizers.

The marker family failed. The work on its execution path did not disappear.

The defensive implications follow from the same evidence: keep user content separate from trusted role/channel structure; authorize normalized actions rather than lexical cues; and inspect the generated call, parsed action, and reconstructed next input separately. Runtime designers should also consider an explicit terminal path when a task's useful action is already complete. These are proposed lessons, not mitigations tested here.

Conclusion and Evidence Boundary

The reusable result is a method, not a universal prompt: connect source-level analysis to model-specific message changes, preserve the scored action, and judge each change by the complete replay.

Local experiments used the competition GGUFs, `llama-cpp-python 0.3.34`, deterministic decoding, an 8,192-token context, and an RTX A5000. Gemma finalists used 24 offloaded layers; the 23-layer language ablation is separate. Local seconds are not hosted-runtime guarantees.

The source, template, parser, and candidate identities are essential to reproduction. Raw replay JSONs are named in the ledger but are not included here; no model replay was rerun to prepare this note. The result audit could not retrieve historical v57, so the supplied code is not byte-authenticated as that selected entry. The private result validates the family, not an independently isolated optimization or hidden guardrail implementation. [3–6]

Acknowledgments. I thank the participants who shared public code and analysis. OpenAI Codex performed much of the source inspection, implementation, experiment orchestration, and trace comparison under my direction. I defined the objectives, constraints, priorities, acceptance gates, and final submission decisions.

References

[1] Kaggle, *AI Agent Security — Multi-Step Tool Attacks*: competition overview and evaluator FAQ. [2] Pilkwang Kim, *Crossing 60 on v3.1.2: a throughput mental model, and a pile of things that didn't work*. [3] *WORKING_NOTE_USER_MESSAGE_PROGRAM.md*. [4] *RESULTADOS_EXPERIMENTOS(2).md*, supported by the technical and model-specific audits. [5] Supplied `attack.py` snapshot. [6] *REPORTE_FINAL_KAGGLE_AI_SECURITY_BETA(2).md*, 1 September 2026. [7] OpenAI, *Harmony response format*. [8] Google, *Gemma 4 Prompt Formatting*.

Public links, artifact identifiers, source hashes, and numerical checks are in the accompanying **Supporting_Evidence.md**.